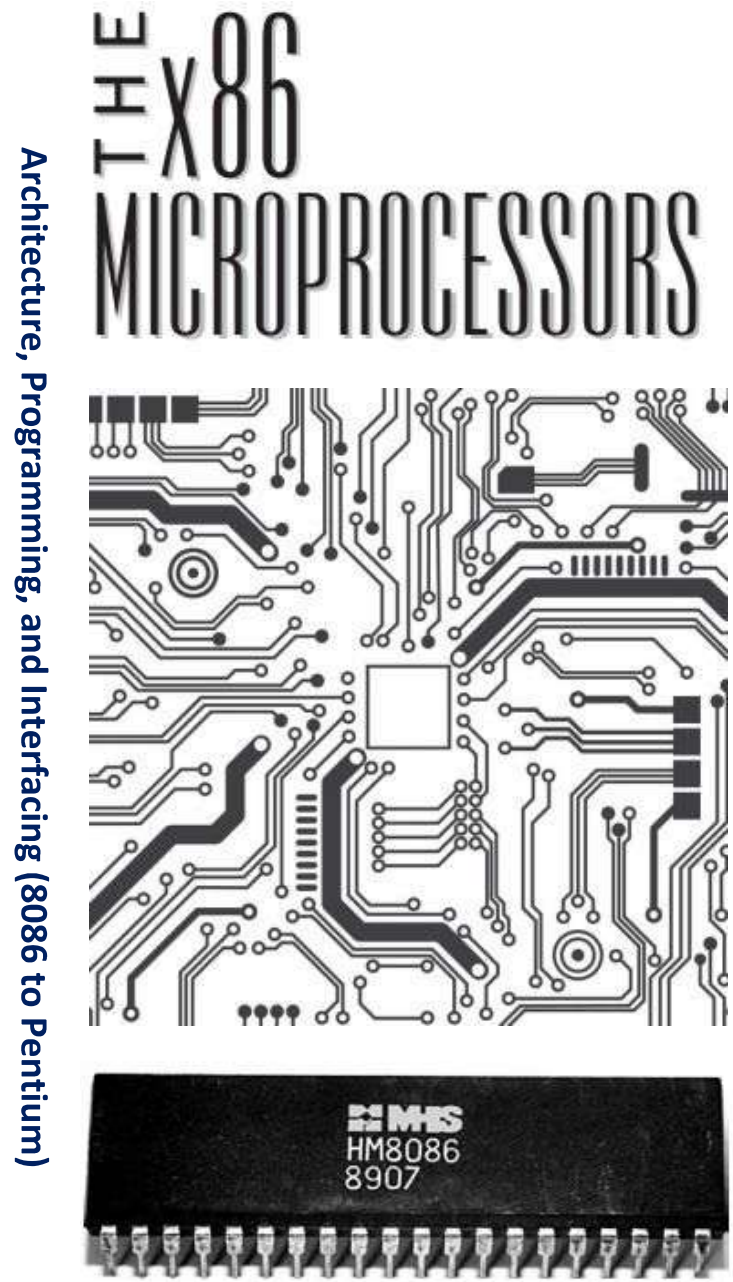
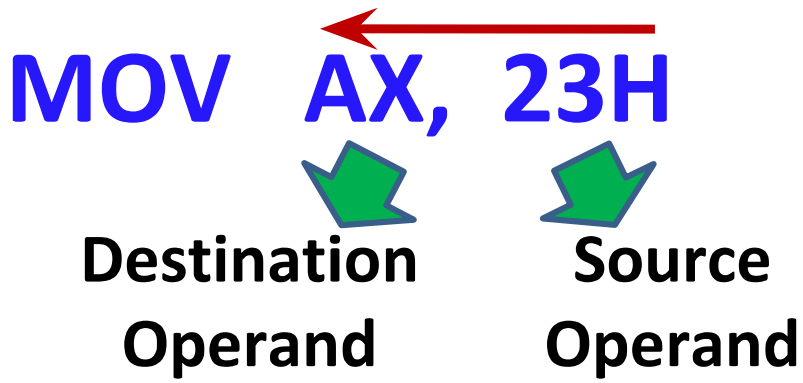
Addressing Mode of 8086

Prof. Dr. Shamim Akhter
Professor, Computer Science and Engineering (CSE)



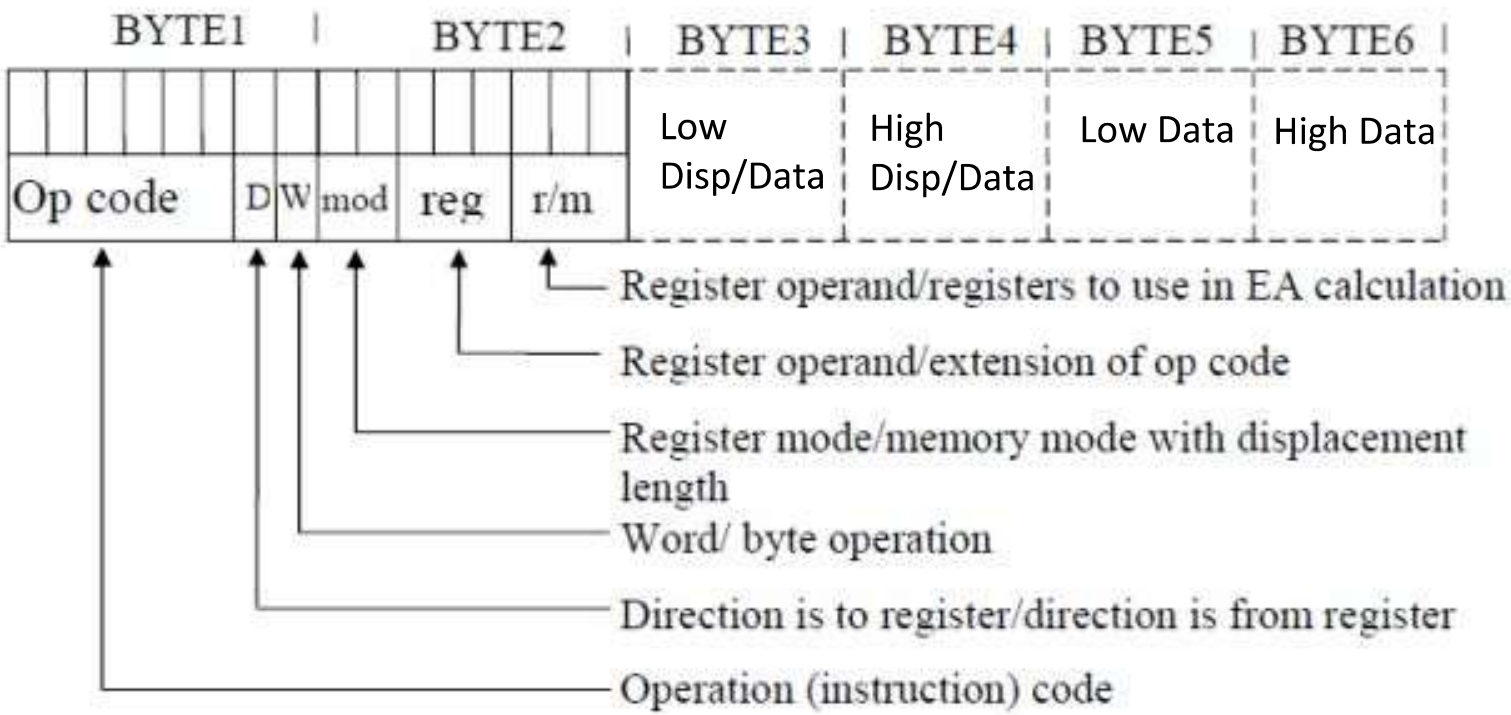
Addressing Mode

- Identify the location and value of an operand.



Register Addressing	Base Plus Index Addressing
Immediate Addressing	Register Relative Addressing
Direct Addressing	Base Relative Plus Index Addressing
Register Indirect Addressing	Scaled Index Addressing
RIP Relative Addressing	

8086 Instruction Format



- **Opcode field (6-bit):** Specifies the operation, such as add, subtract, or move, that is to be performed.
- **Register direction bit (D bit):** Tells whether the register operand specified by register in byte 2 is the source or destination operand. Logic 1 in this bit position indicates that the register operand is a destination operand, and logic 0 indicates that it is a source operand.
- **Data size bit (W bit):** Specifies whether the operation will be performed on 8-bit or 16-bit data. **Logic 0 selects 8 bits and** 1 selects 16 bits as the data size.

- The register filed (reg.) is 3-bit. It is used to identify the register for the first operand, which is the one that was defined as the source or destination by the D bit in byte 1.

REG	W = 0	W = 1
000	AL	AX
001	CL	CX
010	DL	DX
011	BL	BX
100	AH	SP
101	CH	BP
110	DH	SI
111	BH	DI

- The 2-bit mod field and 3-bit r/m field together specify the second operand.

- Encoding for these two fields is shown in the following table.
- Mod indicates whether the operand is in a register or memory.
  - Note that in the case of a second operand in a register, the mod field is always 11. The r/m field, along with the W bit from byte1, selects the register.

MOD	*Function*
00	No displacement
01	8-bit sign-extended displacement
10	16-bit signed displacement
11	R/M is a register

Table of R/m field encoding

MOD = 11			EFFECTIVE ADDRESS CALCULATION			
R/M	W = 0	W = 1	R/M	MOD = 00	MOD = 01	MOD = 10
000	AL	AX	000	(BX) + (SI)	(BX) + (SI) + D8	(BX) + (SI) + D16
001	CL	CX	001	(BX) + (DI)	(BX) + (DI) + D8	(BX) + (DI) + D16
010	DL	DX	010	(BP) + (SI)	(BP) + (SI) + D8	(BP) + (SI) + D16
011	BL	BX	011	(BP) + (DI)	(BP) + (DI) + D8	(BP) + (DI) + D16
100	AH	SP	100	(SI)	(SI) + D8	(SI) + D16
101	CH	BP	101	(DI)	(DI) + D8	(DI) + D16
110	DH	SI	110	DIRECT ADDRESS	(BP) + D8	(BP) + D16
111	BH	DI	111	(BX)	(BX) + D8	(BX) + D16

Note: when r/m = 110 and MOD = 00 direct addressing occurred

R/M Code	*Addressing Mode*
000	DS:[BX+SI]
001	DS:[BX+DI]
010	SS:[BP+SI]
011	SS:[BP+DI]
100	DS:[SI]
101	DS:[DI]
110	SS:[BP]*
111	DS:[BX]

Addressing Modes

- **Data Addressing Mode**

No memory reference to fetch data
Faster processing
Operand is in register

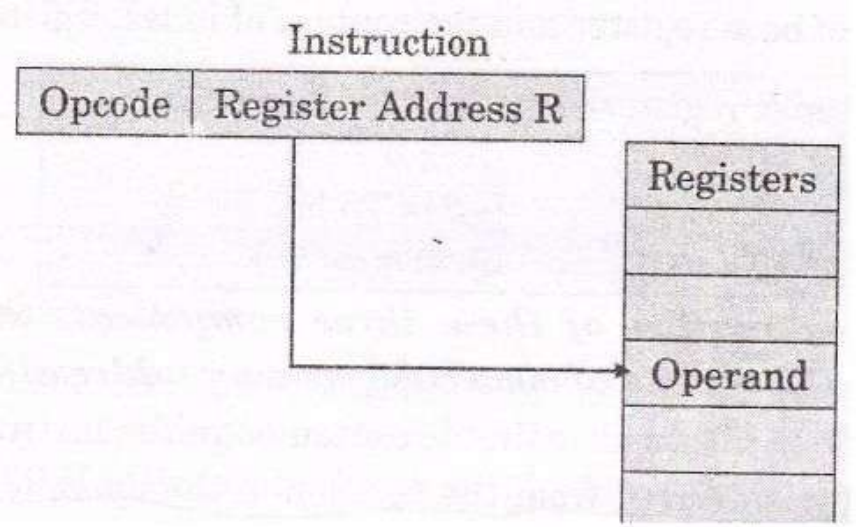
1. Register Addressing

- Transfer a copy of a byte/word from the source register to the destination register
- Operand is in general purpose register

MOV AX, BX
MOV ECX, EBX
MOV RAX, RBX

MOV ES, DS
MOV BL, DX
MOV CS, AX

NOT ALLOWED



MOV Move data

100010dw oorrrmmm disp

Format	Examples	Microprocessor	Clocks

MOV reg,reg	MOV CL,CH
	MOV BH,CL
	MOV CX,DX
	MOV EAX,EBP
	MOV ESP,ESI
		8086	2

		8088	2
		80286	2
		80386	2
		80486	1
		Pentium–Core2	1

MOV mem,reg	MOV DATA7,DL
	MOV NUMB,CX
	MOV TEMP,EBX
	MOV [ECX],BL
	MOV [DI],DH
		8086	9 + ea

		8088	13 + ea
		80286	3
		80386	2
		80486	1
		Pentium–Core2	1

MOV reg,mem	MOV DL,DATA8
	MOV DX,NUMB
	MOV EBX,TEMP+3
	MOV CH,TEMP[EDI]
	MOV CL,DATA2
		8086	10 + ea

		8088	12 + ea
		80286	5
		80386	4
		80486	1
		Pentium–Core2	1

Encode MOV BL, AL

Move Opcode: MOV is 100010

For byte 1:

The six most significant bits of first byte is 100010.

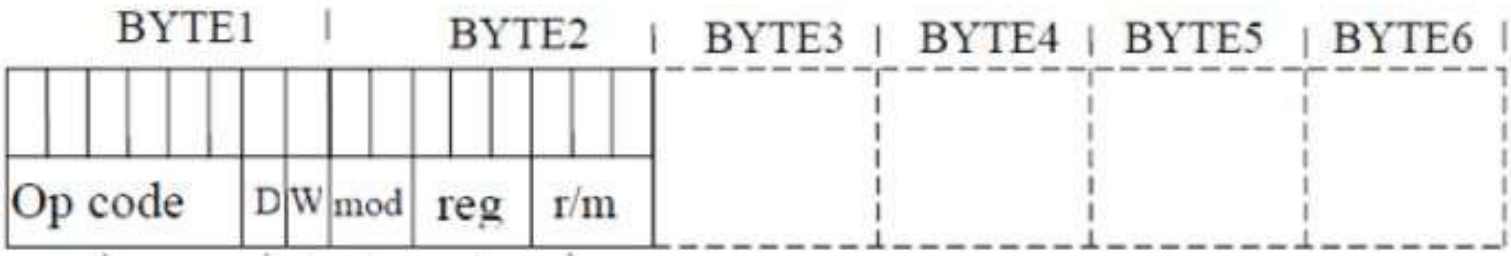
D =0 to specify that a register AL is the source operand.

W=0 to specify an 8-bit data operation.

$$\therefore byte\ 1 = (10001000)_2 = (88)_{16}$$

For byte 2: $(11000011)_2 = (C3)_{16}$

Thus, the hexadecimal machine code for instruction MOV BL,AL=88C3H



Addressing Modes

2. Immediate Addressing

- Immediate implies data immediately follow hexadecimal opcode in the memory
- Operand is constant data and part of instruction

MOV AL, 23H

MOV AX, 3456H

- MOV AX, ‘AB’ -> ASCII characters are stored as BA
- MOV AH,1 not allowed in 64 bit but allowed in 32 and 16 bits

1011wrrr data Format	Examples	Microprocessor	Clocks

MOV reg,imm	MOV BX,22H
	MOV CX,12H
	MOV CL,2
	MOV ECX,123456H
	MOV DI,100
		8086	4

		8088	4
		80286	3
		80386	2
		80486	1
		Pentium–Core2	1
101000dw disp Format	Examples	Microprocessor	Clocks

MOV mem,acc	MOV DATAF,AL
	MOV LIST,AX
	MOV NUMB,EAX
		8086	10

		8088	14
		80286	3
		80386	2
		80486	1
		Pentium–Core2	1

MOV acc,mem	MOV AL,DATAE
	MOV AX,LIST
	MOV EAX,LUTE
		8086	10

		8088	14
		80286	5
		80386	4
		80486	1
		Pentium–Core2	1

Immediate Addressing

MOV opcode 1011

Format:1011wrrr data

- MOV AX, 5001H

  1011 1 000 00000001 01010000

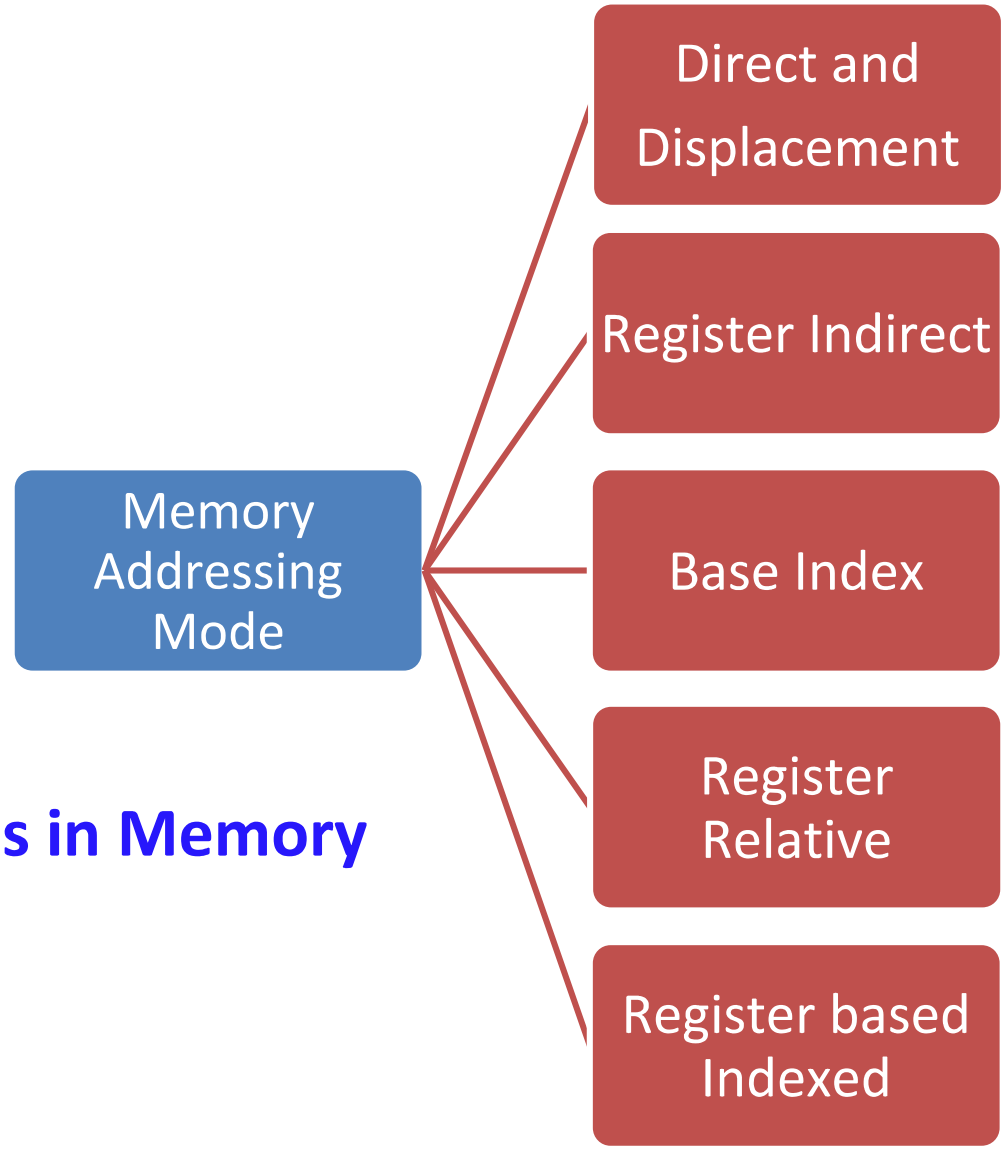
- MOV BX, 23H

  1011 1 011 0010 0011 00000000

- MOV Cl, 23H

  1011 0 001 0010 0011

Memory Addressing Mode



Operand is in Memory

Addressing Modes

- Direct Addressing
  - Operand is the address of the internal data memory

MOV AL, [1234H]
MOV AX, LIST
MOV RAX, HELLO

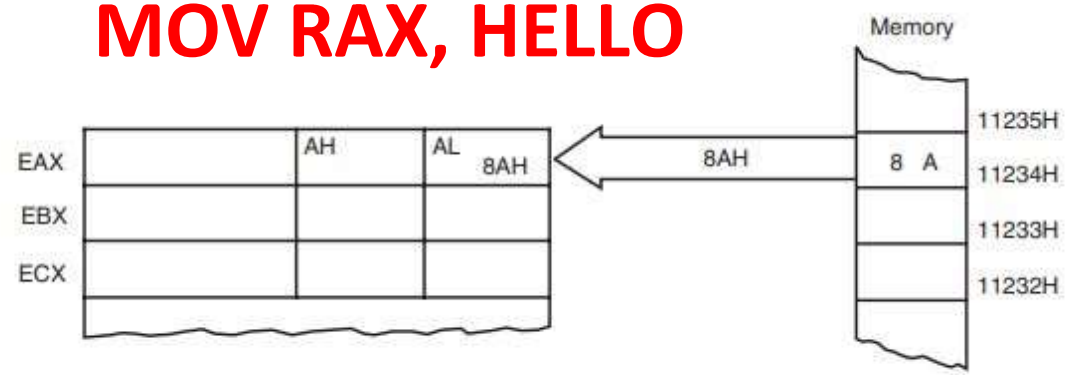
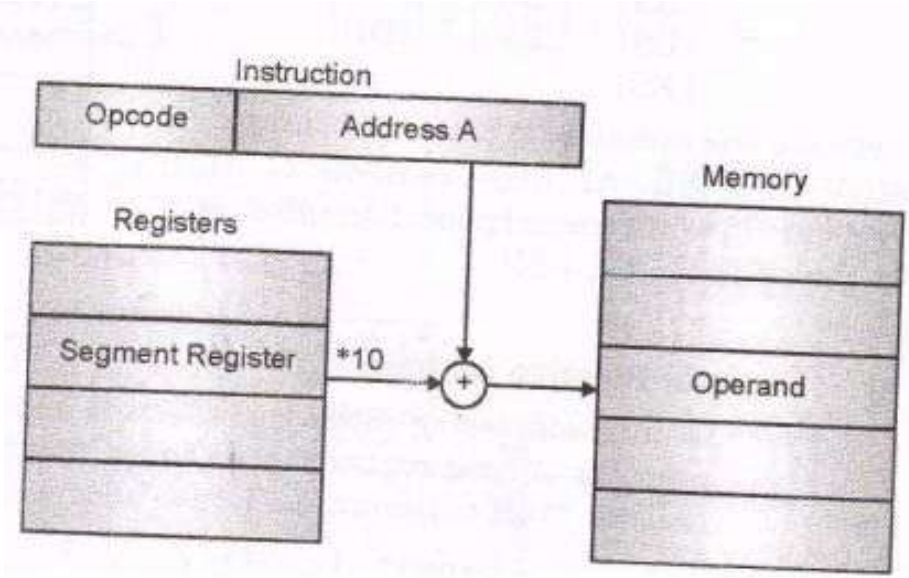


FIGURE 3–5 The operation of the MOV AL,[1234H] instruction when DS = 1000H.



- Displacement Addressing

MOV AL, DS:[1234H]

  - Any segment register can be used.

$$EA = 8/16 \text{ bit displacement}$$

$$PA = \text{segment} : EA$$

$$PA = \begin{Bmatrix} CS \\ DS \\ ES \\ SS \end{Bmatrix} : \begin{Bmatrix} \text{Direct Address} \end{Bmatrix}$$

```
0000 A0 1234 R          MOV AL,DS:[1234H]
0003 BA 0E 1234 R       MOV CL,DS:[1234H]
```

MOV Move data

100010dw oorrrmmm disp

Format	Examples	Microprocessor	Clocks

MOV reg,reg	MOV CL,CH
	MOV BH,CL
	MOV CX,DX
	MOV EAX,EBP
	MOV ESP,ESI
		8086	2

		8088	2
		80286	2
		80386	2
		80486	1
		Pentium–Core2	1

MOV mem,reg	MOV DATA7,DL
	MOV NUMB,CX
	MOV TEMP,EBX
	MOV [ECX],BL
	MOV [DI],DH
		8086	9 + ea

		8088	13 + ea
		80286	3
		80386	2
		80486	1
		Pentium–Core2	1

MOV reg,mem	MOV DL,DATA8
	MOV DX,NUMB
	MOV EBX,TEMP+3
	MOV CH,TEMP[EDI]
	MOV CL,DATA2
		8086	10 + ea

		8088	12 + ea
		80286	5
		80386	4
		80486	1
		Pentium–Core2	1

Direct and Displacement Addressing

MOV instruction format:100010dw oorrrmmm disp
[Not accumulator register]
MOV instruction format:101000dw disp
[Accumulator register]

- **MOV AX, [1234H]**

1010 0011 0011 0100 0001 0010

- **MOV BL, [1234H]**

1000 10 10 00 011 110 0011 1000 0001 0010

```
.data
array DW 1234H
.code
mov BX, array
ret
```

- **MOV BX, [1234H]**
- **1000 10 11 00 011 110 0011 0100 0001 0010**
- **MOV [1234H], BX**
- **1000 10 01 00 011 110 0011 0100 0001 0010**

Addressing Modes

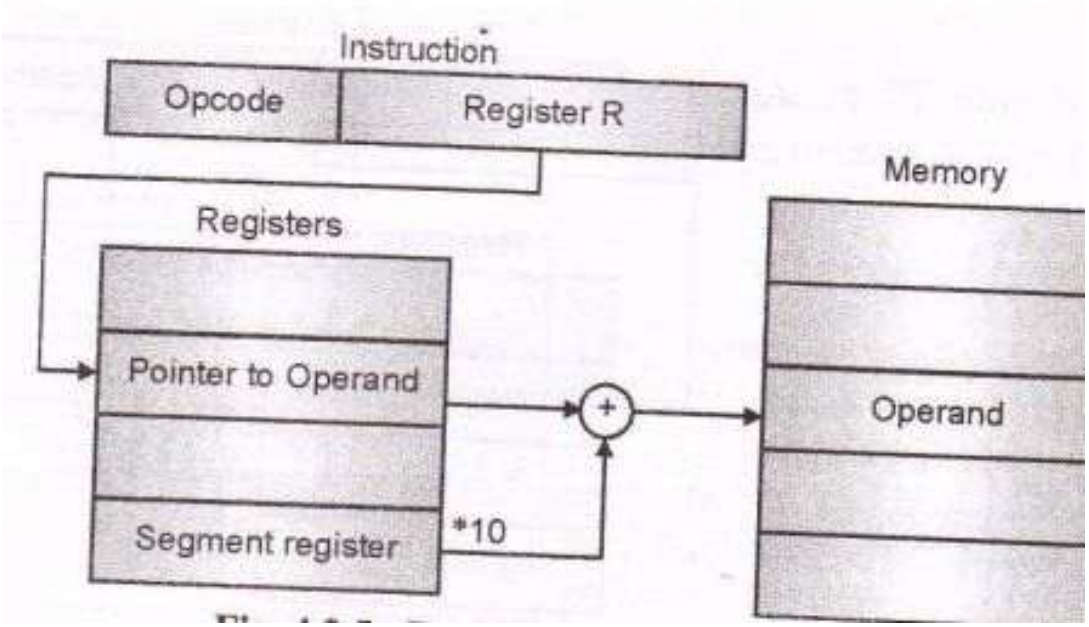
$$EA = \begin{Bmatrix} (BX) \\ (BP) \\ (SI) \\ (DI) \end{Bmatrix} \quad PA = Segment : EA$$

$$PA = \begin{Bmatrix} CS \\ DS \\ SS \\ ES \end{Bmatrix} : \begin{Bmatrix} BX \\ BP \\ SI \\ DI \end{Bmatrix}$$

- Register Indirect addressing
  - Data to be addressed at any memory location through an offset address held any base (BP, BX) or index (SI, DI) register.

Default Segment is Data segment (DS)

MOV AX, [BX]
MOV ECX, [EBX]

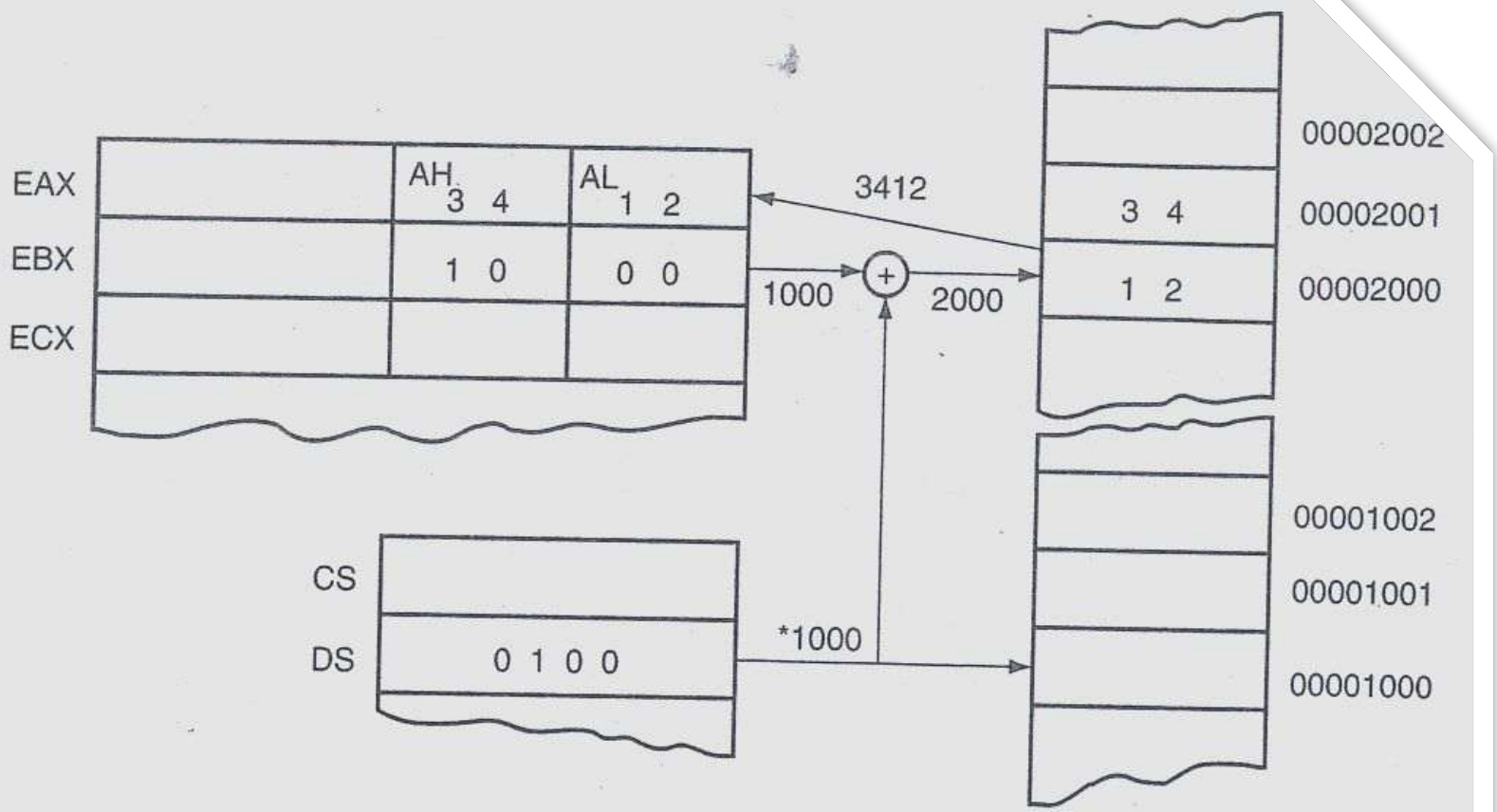


MOV AX, [BP] BP points to stack segment (SS)

MOV [DI], [BX] **Memory-to-memory transfer NOT allowed**

MOV BYTE PTR [DI], 10H Assembler can't determine the location size – byte, word, double word
Thus PTR directives are used

MOV AX, [BX]



*After DS is appended with a 0.

IRE 3–6 The operation of the MOV AX,[BX] instruction when BX = 1000H and DS = 0100H. at this instruction is shown after the contents of memory are transferred to AX.

MOV Move data

100010dw oorrrmmm disp

Format	Examples	Microprocessor	Clocks

MOV reg,reg	MOV CL,CH
	MOV BH,CL
	MOV CX,DX
	MOV EAX,EBP
	MOV ESP,ESI
		8086	2

		8088	2
		80286	2
		80386	2
		80486	1
		Pentium–Core2	1

MOV mem,reg	MOV DATA7,DL
	MOV NUMB,CX
	MOV TEMP,EBX
	MOV [ECX],BL
	MOV [DI],DH
		8086	9 + ea

		8088	13 + ea
		80286	3
		80386	2
		80486	1
		Pentium–Core2	1

MOV reg,mem	MOV DL,DATA8
	MOV DX,NUMB
	MOV EBX,TEMP+3
	MOV CH,TEMP[EDI]
	MOV CL,DATA2
		8086	10 + ea

		8088	12 + ea
		80286	5
		80386	4
		80486	1
		Pentium–Core2	1

MOV format 100010dw oorrrmmm disp

- MOV AX, [BX]

  1000 10 11 00 000 111…

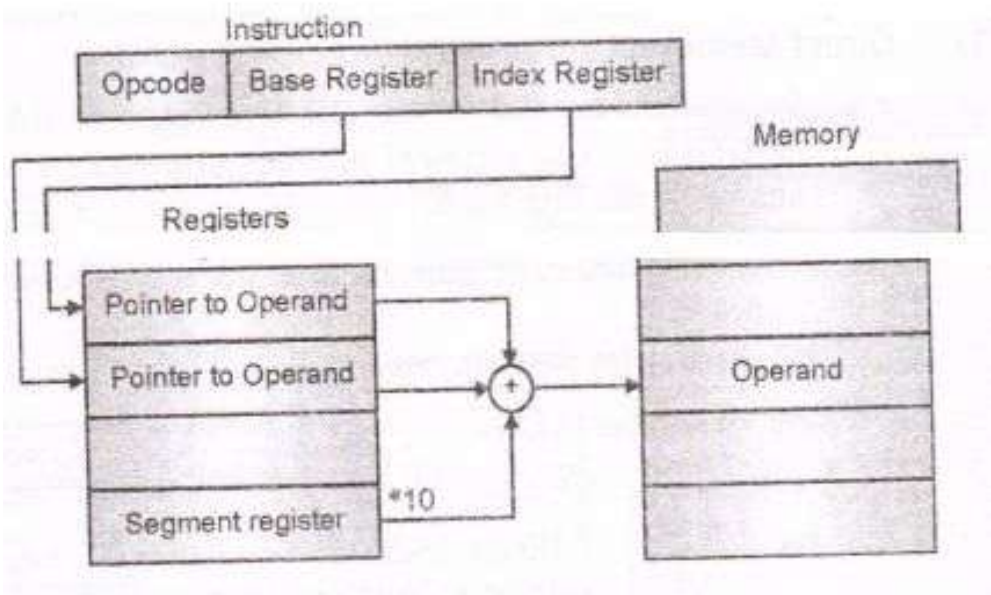
- MOV CX, [BX]

  100010 11 00 001 111…

Addressing Modes

- Base-Plus-Index Addressing
  - Similar to indirect addressing because it indirectly addresses memory data.

MOV DX, [BX+DI]
MOV DL, [EAX+EBX]
MOV DX, [BX][DI]
MOV CH, [BP+SI]
MOV [BX+SI], SP
MOV [EAX+EBX], ECX



$$\therefore EA = \{\text{Base register}\} + \{\text{Index register}\}$$

$$= \begin{Bmatrix}(BX)\\(BP)\end{Bmatrix} + \begin{Bmatrix}(SI)\\(DI)\end{Bmatrix}$$

$$PA = \text{Segment register} : EA$$

$$= \begin{Bmatrix}CS\\SS\\DS\\ES\end{Bmatrix} : \begin{Bmatrix}(BX)\\(BP)\end{Bmatrix} + \begin{Bmatrix}(SI)\\(DI)\end{Bmatrix}$$

MOV ARRAY[DI], BL

Addressing ARRAY data with Register Addressing

```
 .data
array DW 1234H,1243H, 1111H
.code
mov DI, 8
mov BX, array
mov  [BX+DI], BL
ret
```

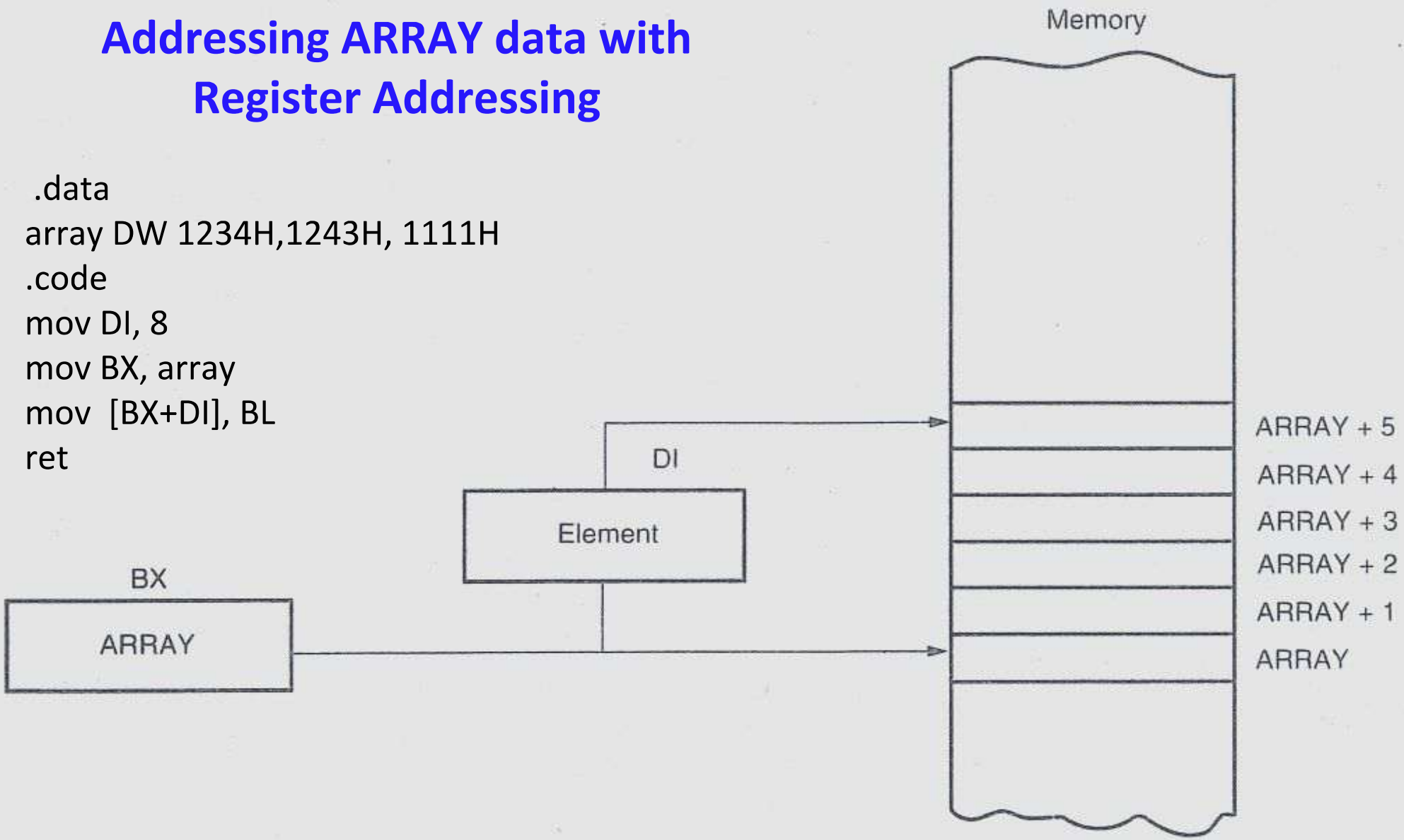


FIGURE 3–9 An example of the base-plus-index addressing mode. Here an element (DI) of an ARRAY (BX) is addressed.

MOV DX, [BX+DI]

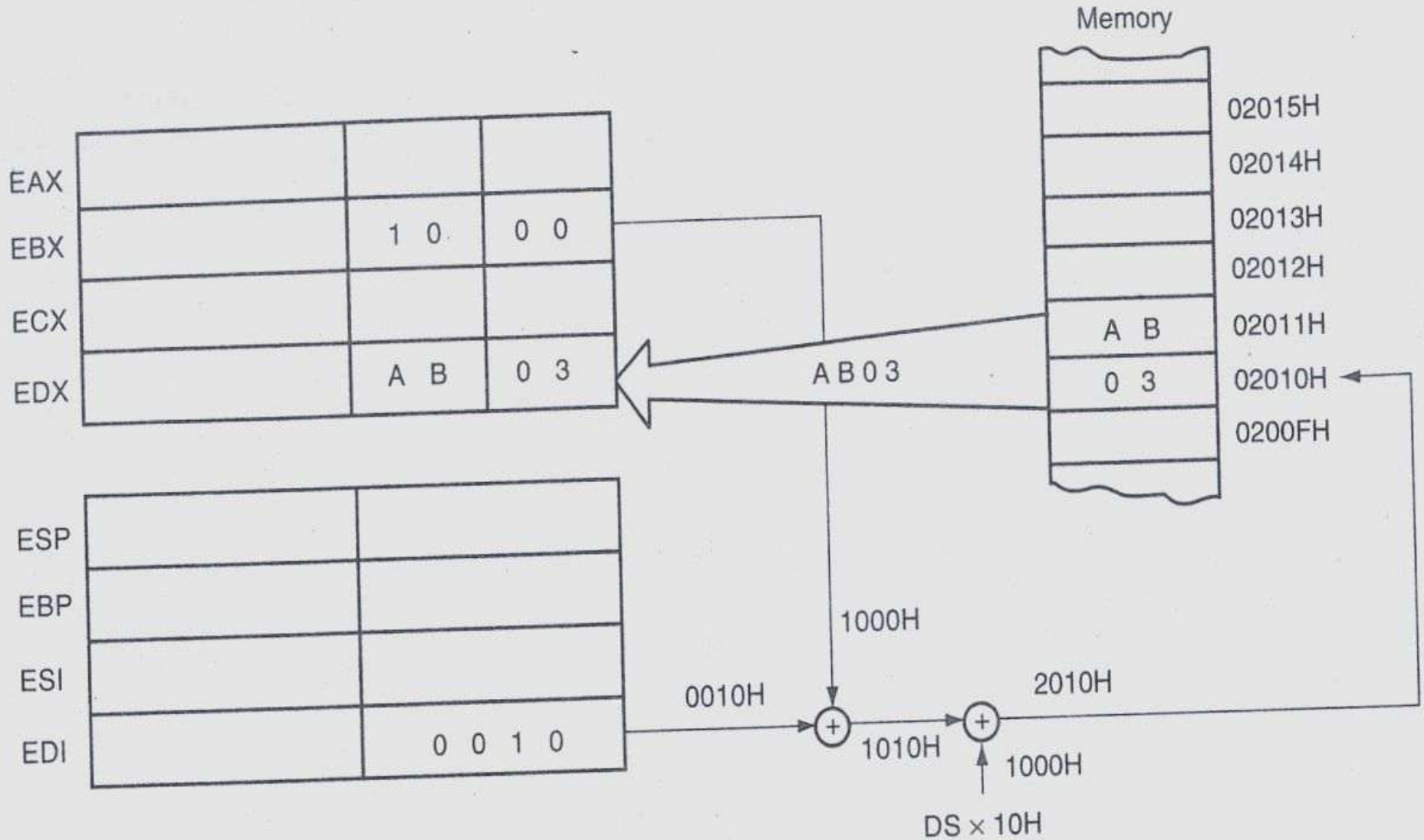


FIGURE 3–8 An example showing how the base-plus-index addressing mode functions for the MOV DX,[BX+DI] instruction. Notice that memory address 02010H is accessed because DS = 0100H, BX = 100H, and DI = 0010H.

MOV format 100010dw oorrrmmm disp

- MOV DX, [BX+DI]
  100010 11 00010001…

```
 .data
array DW 1234H,1243H, 1111H
.code
mov DI, 8
mov BX, array
mov DX, [BX+DI]
ret
```

MOV [BX+DI], BL
100010 00 00 011 001…

```
 .data
array DW 1234H,1243H, 1111H
.code
mov DI, 8
mov BX, array
mov  [BX+DI], BL
ret
```

Addressing Modes

- Register Relative Addressing
  - Similar to base-plus-index addressing and displacement addressing.

MOV AX, [BX+1000H]

8-bits displacement
(sign extended) 16 bits displacement
(sign extended) 32 bits displacement

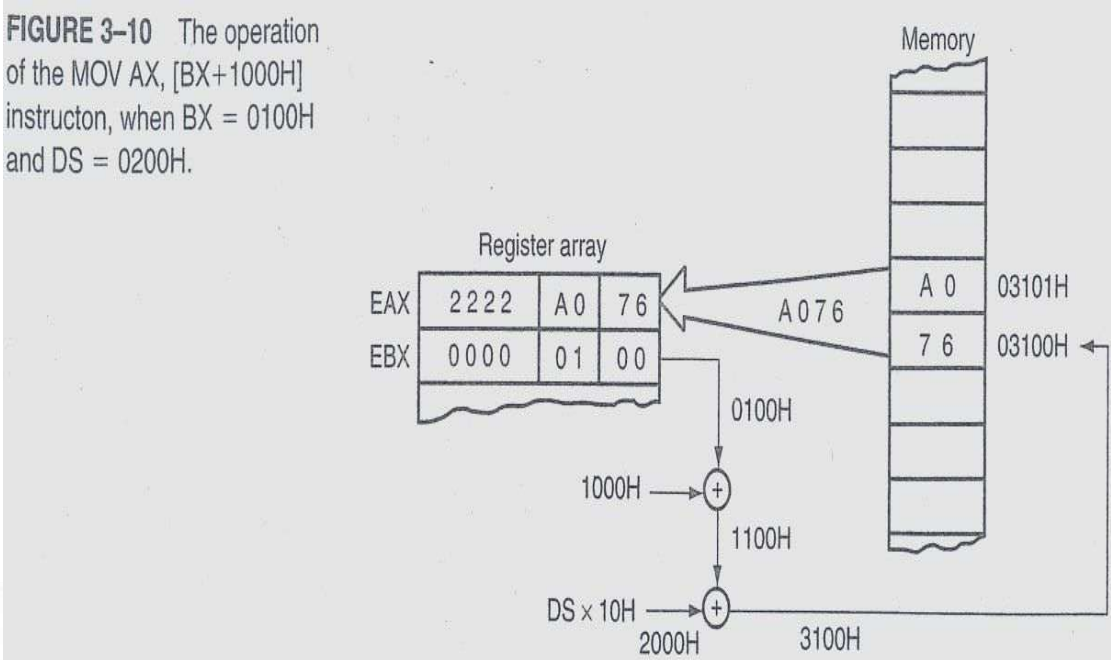
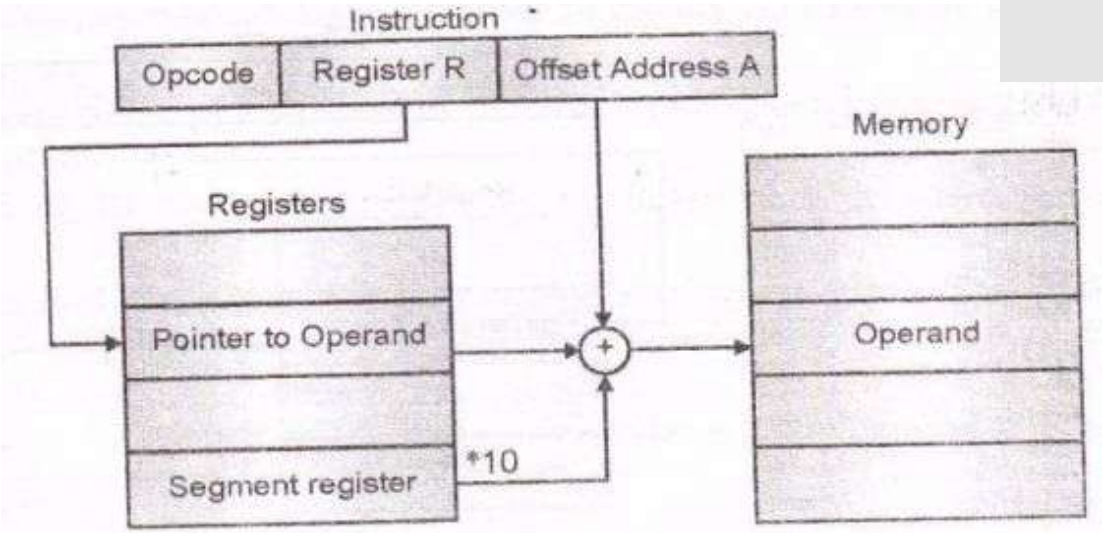


FIGURE 3–10 The operation of the MOV AX, [BX+1000H] instructon, when BX = 0100H and DS = 0200H.



$$EA = \begin{Bmatrix} (BX) \\ (BP) \\ (SI) \\ (DI) \end{Bmatrix} + \begin{Bmatrix} \text{8 bitdisplacement} \\ \text{(sign extended)} \\ \text{16 bit displacement} \end{Bmatrix}$$

$$PA = \text{Segment} : EA = \begin{Bmatrix} CS \\ ES \\ DS \\ SS \end{Bmatrix} : \begin{Bmatrix} (BX) \\ (BP) \\ (SI) \\ (DI) \end{Bmatrix} + \begin{Bmatrix} \text{8/16 bit} \\ \text{offset} \end{Bmatrix}$$

MOV format 100010dw oorrrmmm disp

- MOV DX, [BX+08H]
  100010 11 01010111 00001000

```
 .data
array DW 1234H,1243H, 1111H
.code
mov DI, 8
mov BX, array
mov DX, [BX+08H]
ret
```

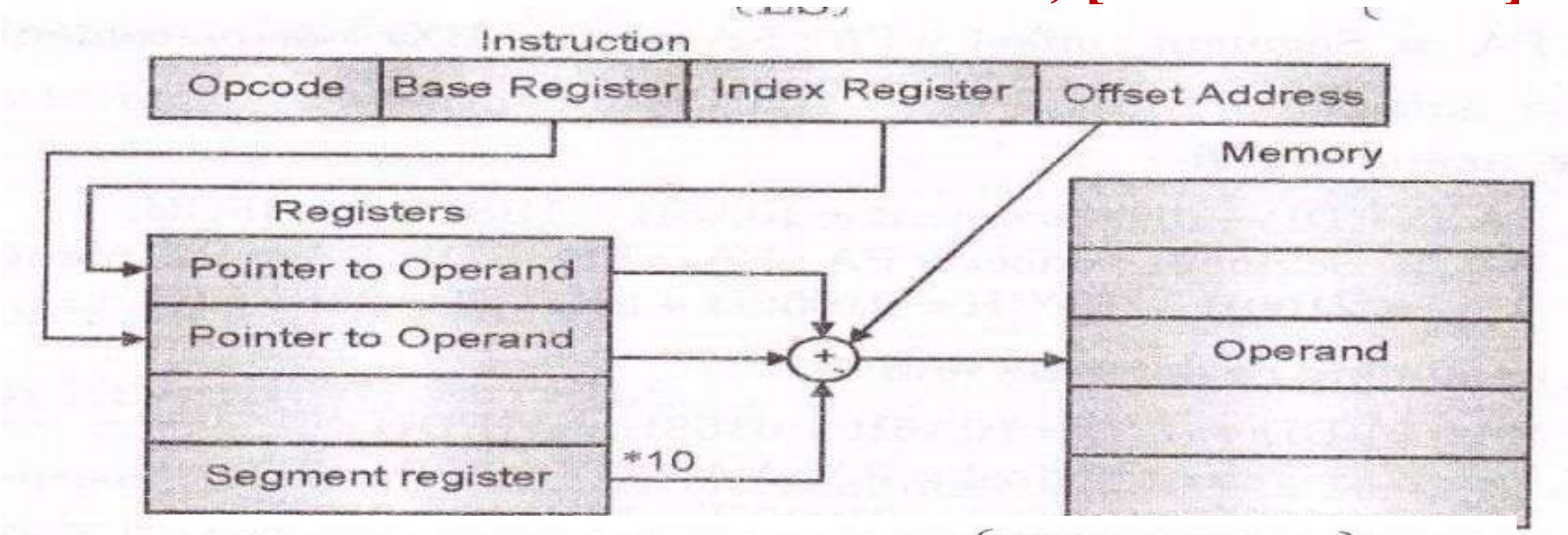
Examples of register relative addressing

Assembly Language	Size	Operation
MOV AX, [DI+100H]	16 bits	Copies the word contents of the data segment memory location addressed by DI plus 100H into AX
MOV ARRAY[SI], BL	8 bits	Copies BL into the data segment memory location addressed by ARRAY plus SI
MOV LIST[SI+2], CL	8 bits	Copies CL into the data segment memory location addressed by the sum of LIST, SI and 2
MOV DI, [EAX+10H]	16 bits	Copies the word contents of the data segment memory location addressed by EAX plus 10H into DI
MOV ARRAY[EBX], EAX	32 bits	Copies EAX into the data segment memory location addressed by ARRAY plus EBX
MOV ARRAY[RBX], AL	8 bits	Copies AL into the memory location ARRAY plus RBX (64-bit mode)

Base Relative Plus Index Addressing

- Similar to base-plus-index addressing, but it adds displacement

MOV AX, [BX+SI+100H]



$$EA = \{\text{Base register}\} + \{\text{Index register}\} + \left\{\begin{array}{c}\text{8 bit displacement}\\ \text{(sign extended)}\\ \text{16 bit displacement}\end{array}\right\}$$

$$= \left\{\begin{array}{c}(BX)\\(BP)\end{array}\right\} + \left\{\begin{array}{c}(SI)\\(DI)\end{array}\right\} + \left\{\begin{array}{c}\text{8/16 bit}\\ \text{displacement}\end{array}\right\}$$

$$PA = \text{Segment register} : EA = \left\{\begin{array}{c}CS\\SS\\DS\\ES\end{array}\right\} : \left\{\begin{array}{c}(BX)\\(BP)\end{array}\right\} + \left\{\begin{array}{c}(SI)\\(DI)\end{array}\right\} + \left\{\begin{array}{c}\text{8/16 bit}\\ \text{displacement}\end{array}\right\}$$

Instruction

MOV AX, [BX+SI+100H]

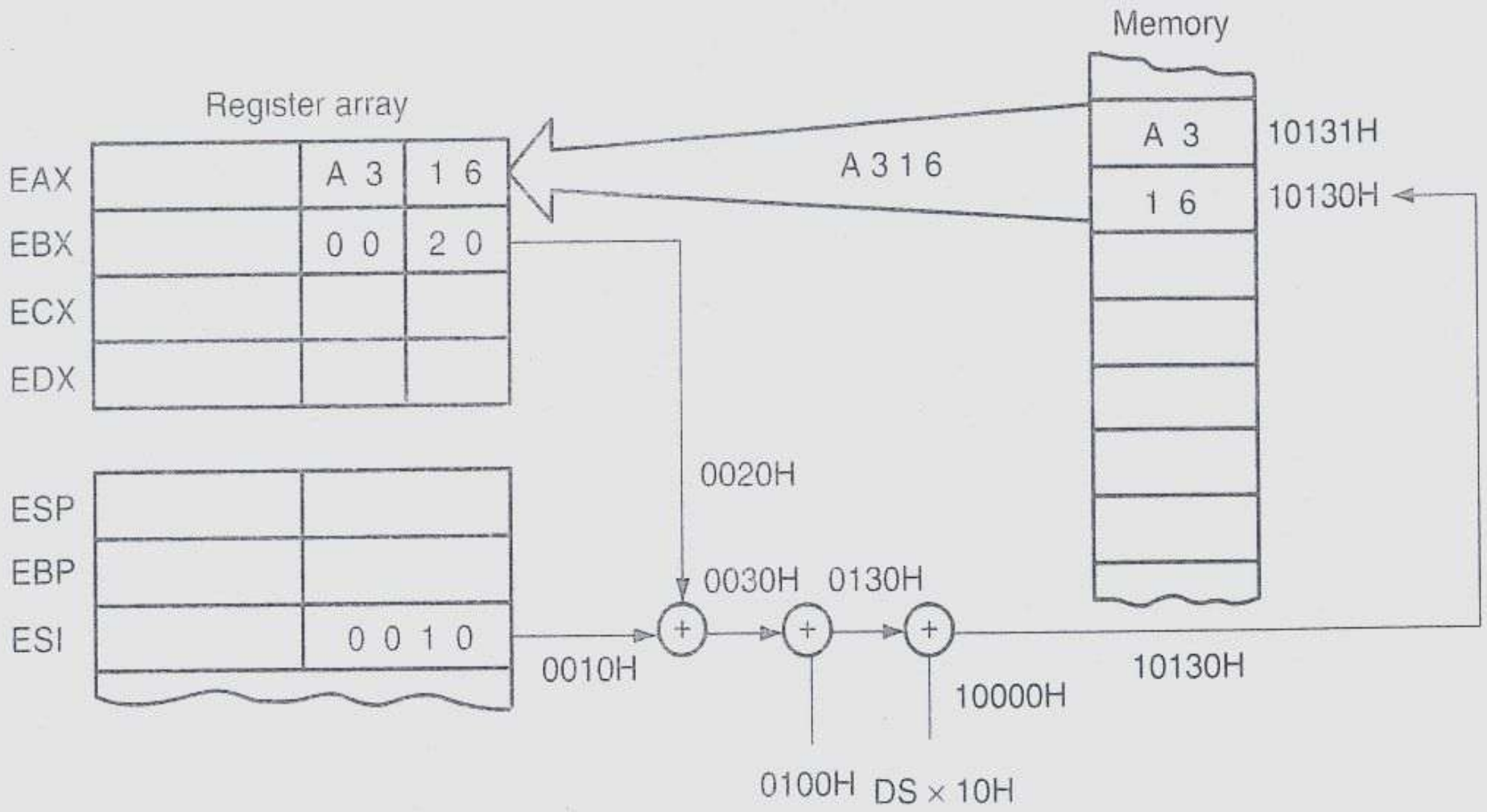
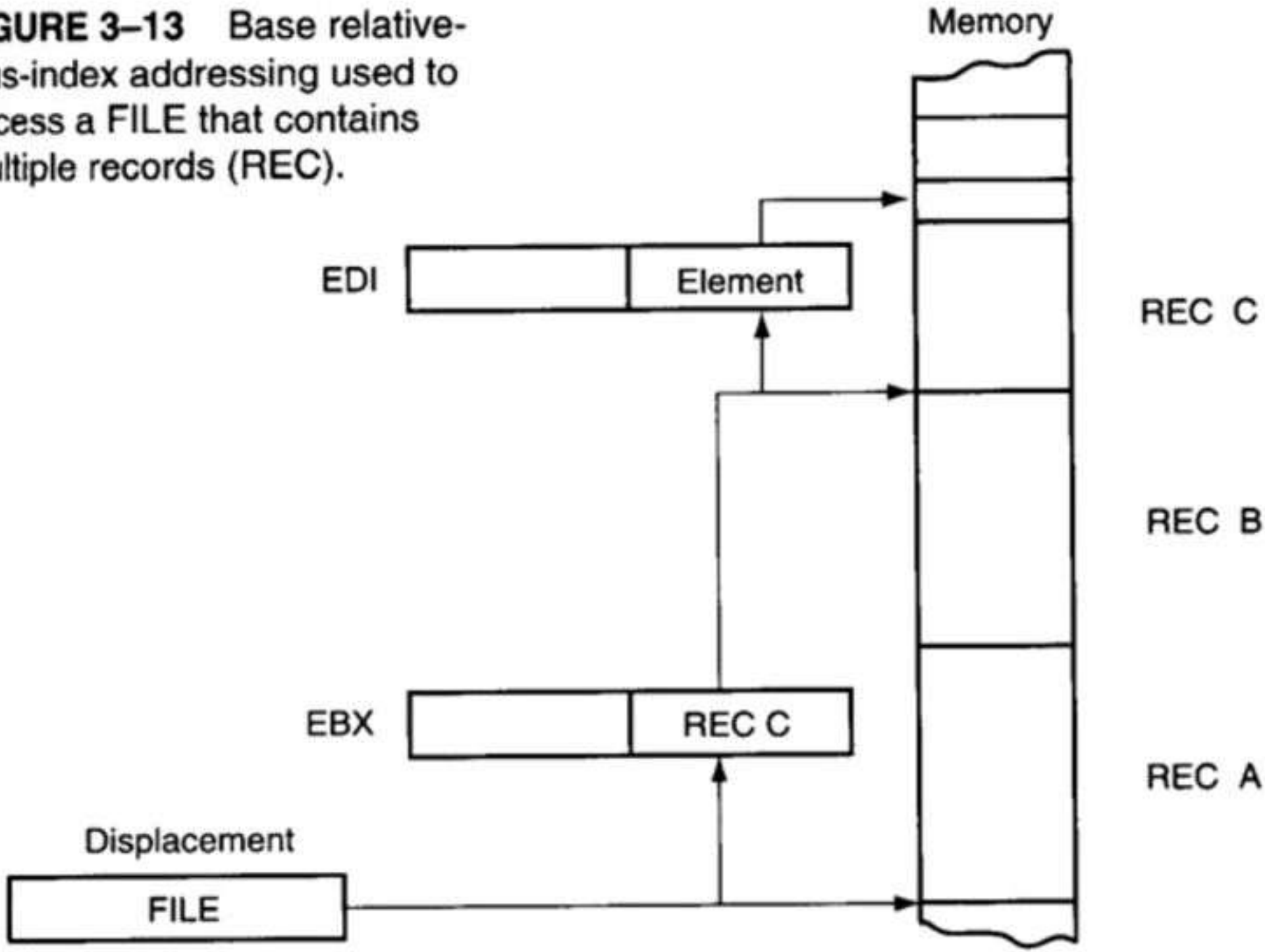


FIGURE 3–12 An example of base relative-plus-index addressing using a MOV AX,[BX+SI+100H] instruction. Note: DS = 1000H.

FIGURE 3–13 Base relative-plus-index addressing used to access a FILE that contains multiple records (REC).



mov DX, [BX+DI+3H]

- MOV format 100010dw oorrrmmm disp
- 100010 11 01 010 001 00000011

```
 .data
array DW 1234H,0000H, 0000H
.code
mov DI, 1H
mov BX, array
mov DX, [BX+DI+3H]
ret
```

TABLE 3–8 Example base relative-plus-index instructions.

Assembly Language	*Size*	*Operation*
MOV DH,[BX+DI+20H]	8 bits	Copies the byte contents of the data segment memory location addressed by the sum of BX, DI and 20H into DH
MOV AX,FILE[BX+DI]	16 bits	Copies the word contents of the data segment memory location addressed by the sum of FILE, BX and DI into AX
MOV LIST[BP+DI],CL	8 bits	Copies CL into the stack segment memory location addressed by the sum of LIST, BP, and DI
MOV LIST[BP+SI+4],DH	8 bits	Copies DH into the stack segment memory location addressed by the sum of LIST, BP, SI, and 4
MOV EAX,FILE[EBX+ECX+2]	32 bits	Copies the doubleword contents of the memory location addressed by the sum of FILE, EBX, ECX, and 2 into EAX

Addressing Modes

- Scaled-Index Addressing(80386 to above)
  - Two 32-bit registers (Base and Index)
    - 2nd register (index) is multiplied by a scaling factor
    - Scaling factor can be 1x, 2x(word), 4x(double word), or 8x (quadword)

MOV AX, [EDI+2*ECX] --word addressing

ECX has 00000001H increase ECX by scaled 2

MOV EAX, ARRAY [4*ECX] ---Double word addressing

Addressing Modes

- **RIP Relative Addressing**
  - Uses 64-bit instruction pointer register in the 64-bit mode to address a linear location in the flat memory model.